CANDY
DIALOGUE ENGINE

Basic Setup Guide

1.1.0

Control commands can be classified in two categories: commands that modify variables, and commands that execute pre-written code.

For non-coders, such as dialogue writers: you don't need to actually know how to code to use these commands, but you do need to understand a few basic concepts, such as what functions and signals (and their respective arguments) are, and the syntax for modifying variables (which is like writing a simple math equation, not coding).

Variable Commands

§Set Command

The §Set command can change the value of any variable in your project. This is done by writing a variable reference, an operator and an expression, like so:

£globals.health = 10

£globals.gold += 12

Variable

The Variable field contains the reference to the variable you want to modify. This reference must be entered using the Candy DE variable reference format: that is, it must be prefixed by either the **singleton_symbol**, **the node_symbol** or the **vardict_symbol** (default '£', '$' and '€' respectively), like so:

£globals.variable

See the **Variables** guide for further details.

Operators

Operators are math symbols, methods or functions that indicate what the dialogue engine should do with a variable and an expression's final value. A few basic examples:

=	→	Set the variable to the expression's value.
+=	→	Add the expression's value to the variable.
-=	→	Subtract the expression's value from the variable.

***= or ×=** → Multiply the variable by the expression's value.

/= or ÷= → Divide the variable by the expression's value.

The (⁝) button will display a small list of the most commonly used operators, for convenience. However, many more operators are supported by Candy DE, if you type them manually. The full list of operators is found in the **Operator List** PDF file.

If an operator is not supported by Candy DE, you can get around this by using the '=' operator, and including the variable to modify into the expression. For example:

£globals.variable | = | £globals.variable *unsupported_operator* 4 * (3+2)

Expression

The Expression field is where you must write the value you want to modify the variable by. For example, if the operator is '÷=' and the expression is 2, the variable will be divided by 2.

However, expressions don't need to be a single value. For example, you could write:

2 + 3 - 1

Parentheses are also supported. For example:

3 * (10 + 2)

The §Set function uses Godot's built-in 'Expression' method to process the expression. This means that you should be able to write any variable assignment that you could normally write in GDScript. Even built-in methods are supported, such as:

floor(3.14)

Custom functions should work too, provided you point to them correctly. Use the 'self' prefix for functions inside the **Candy_Engine.gd** script, or an autoload prefix or a path for other functions:

self.candy_to_array(candy_de.flags)

globals.your_function()

$"Node/Path".your_function()

You can of course use variables as values, for example:

globals.damage - £globals.armor

Nesting dictionaries and arrays also works, of course. You can write variables such as:

globals.player["health"]

Super Variable Symbols

For both the Variable field and the Expression field, you can replace the entire field by a variable, using the 'super' variable symbols (default: '££', '$$' and '€€'). See the **Variables** guide for details.

Note that the Operator field uses the standard variable symbols since it doesn't expect to find a variable there.

§Flag Command

The §Flag command is nearly identical to the §Set command, with one difference: it's used to quickly change the value of a progression flag located in the **flags** dictionary, in **Candy_Database.gd**.

To change a flag with the §Set command, you'd need to write '£candy_de.flags["flag_1"]' in the F field. With the flag command, you can directly write 'flag_1', without symbols or prefixes, in the **Flag** field.

It's basically a specialized and simplified version of §Set, to save time.

Note that in the **Flag** field, you can use the standard variable symbols (default: '£', '$' and '€') to provide a flag name stored inside a variable. Unlike the §Set command's Variable field, the Flag field doesn't require the 'super' symbols.

The §Flag command also calls the **x_flag_changed()** hook function in **Candy_Functions.gd**. This can be used to trigger custom code when a flag is changed (e.g. to update quest trackers).

Everything else is similar to the §Set command.

Note also that the §Flag command remains compatible if you want to use a custom dictionary of your own for progression flags. See the **Database** guide for instructions.

§Name Command

By default, the §Name command is used to change the Display Name of an actor in the actors_dictionary. However, despite its name, it can actually modify any property (key) of an actor in the **actors** dictionary in **Candy_Database.gd**, provided that key accepts a string as a value (other variable types won't work).

In the **Reference** field, enter an actor reference from your **actors** dictionary (e.g. "Alice", without quotation marks).

In the **Name** field, enter the new display name you want to show on screen when the actor speaks (or if you're modifying another key, enter the value you want to assign). Remember that only strings will work.

The **Actor Key** field is only used if you want to modify another actor property than **"Display Name"** in the **actors** dictionary. For example, if you enter 'Gender' in the Actor Key field, the command will modify the actor's **"Gender"** key instead of the **"Display Name"** key. Leave the Actor_Key field empty to modify the **"Display Name"** key by default.

The **Translation Table** field is used to tell the §Name command which translation dictionary to use, in order to translate values for the Actor Key in the different languages your game supports.

The translation dictionary should be provided in either of the following ways:

- If the dictionary is found inside the **Localizations.gd** script, typing the name of the dictionary is sufficient (e.g. '**display_names**').
- If the dictionary is located elsewhere, you need to provide it in the same form as a variable reference. For example: '£globals.dictionary'.
- If you want to provide an actual reference to a variable, so that 'Actor_Key' is assigned the value of that variable, you must use the 'super' variable symbols.
  For example: '££globals.which_dict_to_use'

Note that you only need to provide a translation dictionary name if you are modifying an actor key other than **"Display Name"**: if the command is configured to modify **"Display Name"** and 'Translation Table' is empty, it will use the 'display_names' dictionary in **Localizations.gd** by default.

If values for the current language don't exist in the selected translation dictionary, the command will assign the value as written in the command's Name field.

§Role Command

The §Role command can be used to assign an actor to a role.

In the **Role** field, enter a role reference from the **roles** dictionary in **Candy_Database.gd**. In the **Reference** field, enter an actor reference from the **actors** dictionary.

Remember that roles must always start with the role_symbol (**Candy_Properties.gd**). By default, that symbol is '°'.

§Disposition Command

The §Disposition command can be used to change the disposition(s) of an actor.

In the **Reference** field, enter an actor reference from your **actors** dictionary in **Candy_Database.gd** (e.g. "Alice", without quotation marks).

In the **Disposition** field, enter one or more dispositions (separate by commas ','). See the **Dispositions** guide for more information.

Function/Signal Commands

§Call & §Emit Commands

§Call and §Emit can respectively call a function or emit a signal.

Variable

For §Call, in the **Variable** field: if the called function returns a value, write a references to the variable you want to store the value in. Remember to use the variable symbols ('£', '$' or '€' by default -- see the **Variables** guide for details). If you want to fetch a variable reference from a variable at runtime, use the 'super' symbols (default '££', '$$' or '€€').

For example:

Standard symbols:
Variable = £globals.storage_var → The value returned by the function will be stored in globals.storage_var

Super symbols:
var variable_ref = "£globals.storage_var"
Variable = ££globals.variable_ref → Candy DE will lookup globals.variable_ref, fetch '£globals.storage_var' as if it had been written in the Variable field, and store the returned value in globals.storage_var

Function/Signal

For both commands: in the **Function** or **Signal** field, write the name of the function or signal like you would a variable, using the variable symbols as well. If you want to fetch a variable reference from a variable at runtime, use the 'super' symbols.

Arguments

In the **Arguments** field, write arguments, separated by a comma (,). Arguments can be strings, integers, floats, booleans or variable references (prefixed with the Candy DE variable symbols - e.g. '£', '$' or '€' by default).

Integers, floats and booleans can be written as-is, without special formatting. For example:

3, 4.6, true → will be treated respectively as integer, float and boolean.

Variable references can also be written as-is, for example:

£candy_de.actors['Alice'], $node_path.dialogue_mode

If any arguments are a string, you should wrap them in single quotes ('...'):

'Pause', '30', 'false'

The single quotes are actually optional, and only serve to ensure string arguments aren't confused with other variable types, but it's good practice to always use the quotes. Double quotes should be safe, but single quotes are recommended as a precaution.

If you want to fetch the entire Arguments string from a variable at runtime, use the 'super' variable symbols:

var argument_string = "1, 2, 3"
→ ££globals.argument_string
→ Will be the same as writing 1, 2, 3 in the Arguments field.

Await

Lastly, for the §Call function, you can write 'true' or 'false' in the **Await** field, depending on whether you want to wait for the function to finish before resuming dialogue.

§Await Command

The §Await command pauses dialogue until a specified signal is emitted by a script or node.

If you want to store the signal's arguments in a variable: in the **Variable** field, write a reference to a variable, using the variable symbols. Use the 'super' symbols to fetch a variable reference at runtime (see §Call command above for details).

In the **Signal** field, write a reference to the signal to wait for, prefixed with the Candy DE variable symbols.

For example:

£globals.signal_name

Once again, to fetch a signal reference from a variable at runtime, use the 'super' symbols instead.

§Export & §Import Commands

The §Export and §Import commands are used for exporting variables to a text file, or importing variables from such a file.

In the **Path** field, provide a path on disk to the folder where the file is located.

In the **File** field, write the name of the file (without extension).

In the **Extension** field, write the extension of the file.

In the **Variables** field, write the variables to save or load, separated by a comma (,), in the standard variable reference format.

For the §Export command, in the **Method** field, select what should happen if the specified file already exists:

Overwrite: deletes the existing files, and saves the variables to a new one.

Skip: doesn't save the variables. The command has no effect.

Duplicate: creates a new file, with a number suffix to its name.

Timestamp: saves a new file, with a timestamp suffix to its name.